

No.6-定时器简单理解与使用

由 CNPP

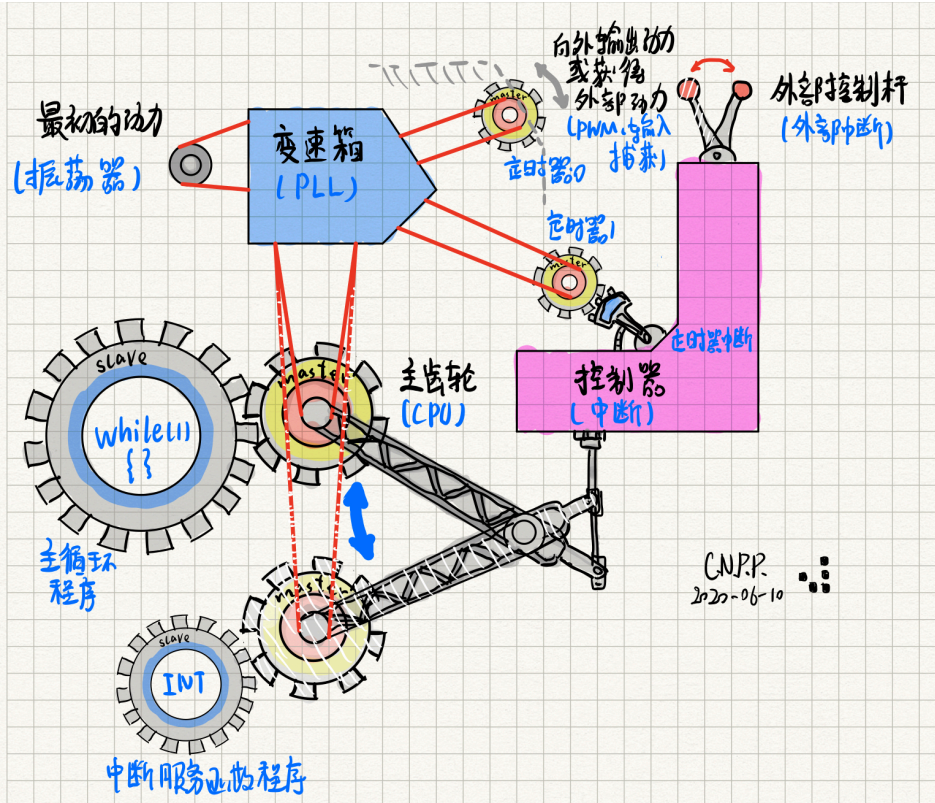
ID:1225

<https://www.emoe.xyz/bonjour-stm32-timer-cookbook/>

May 19, 2020

分类目录: Bonjour STM32, 嵌入式, 所有文章, 硬件

标签: Bonjour STM32, STM32, 教程, 电子入门教程



本文目录

No.6-定时器简单理解与使用

- 0. 前言
- 1. 什么是定时器
- 2. STM32定时器介绍
- 3. 实例：定时器中断
- 4. 实例：定时器输出PWM
- 6. 后记

No.6-定时器简单理解与使用

作者 日期 工具

CNPP 2020-06-10 STM32开发环境，任意带有按键和灯的STM32小板

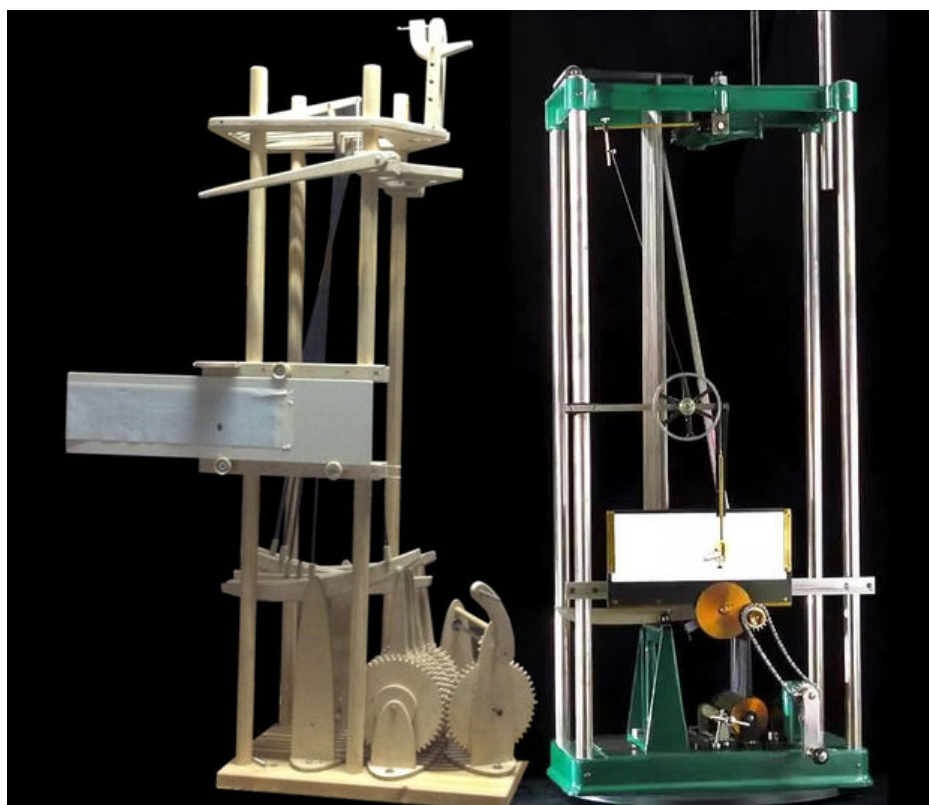
0. 前言

在这一篇中，我们将一起认识单片机的灵魂——定时器。它是每一款单片机都拥有的外设；它原理简单，却不失精妙，可以衍生出丰富多彩的功能。可以说，掌握了定时器的各种灵活应用，才能领会单片机编程的精髓。本篇讲解的定时器知识，不仅仅适用于STM32单片机，更包含有所有单片机通用的思想。接下来我会花一些篇幅和大家聊聊定时器的精妙之处，若想直接参考例程，可转至第3节。

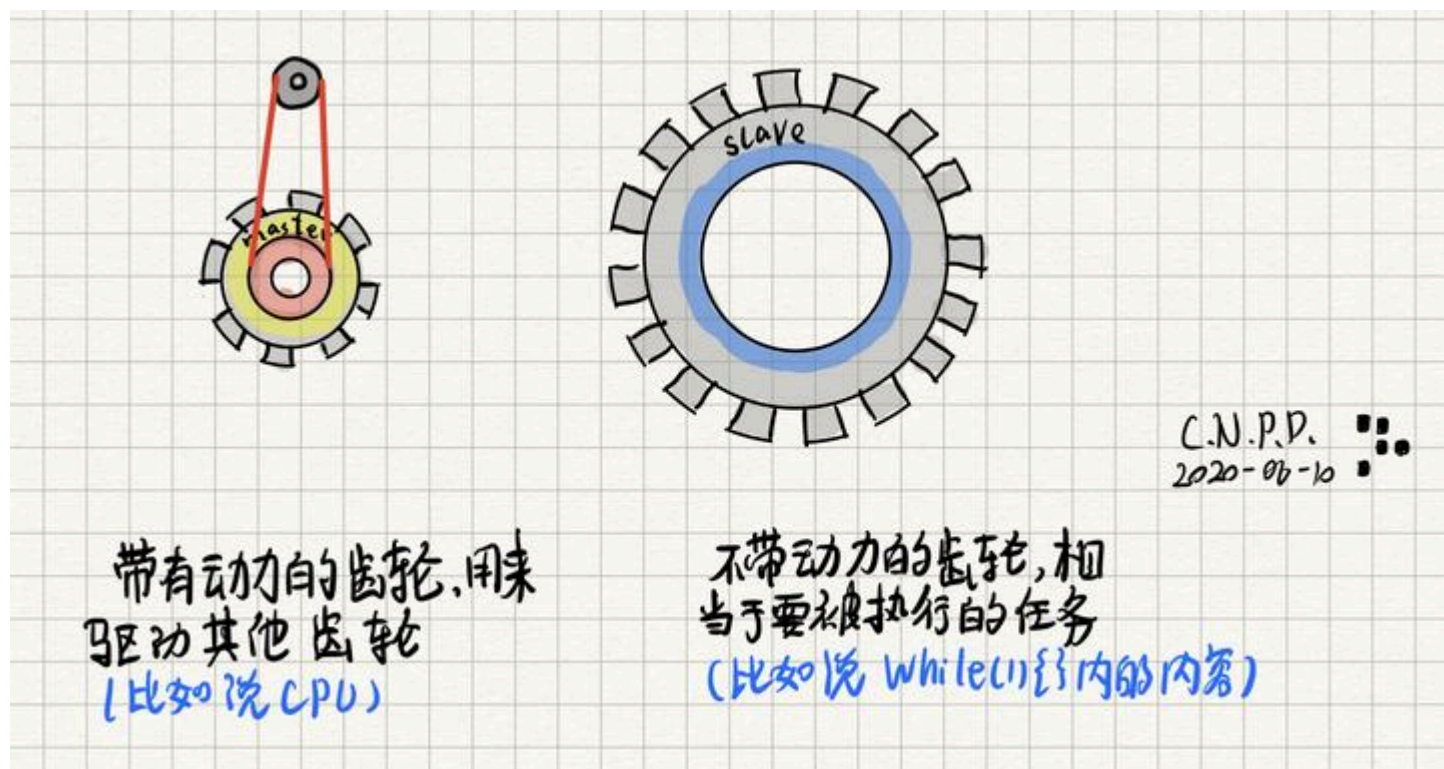
1. 什么是定时器

坦率来说，认识定时器之前，你需要了解不少前置知识，比如单片机是怎样循环运行程序的？中断是什么原理？时钟树是个什么东西？啊啊，其实如果你不太清楚也没有关系，我来带你再换个角度看一看：

其实学习单片机，回想一些先前的技术是很能帮助理解的。比方说，回想到晶体管计算机、电子管计算机，那么再往前一些，甚至手摇计算器，或者“恩尼格玛”密码机？当你看着while(1){}中一行行将要被无尽循环的代码时，思绪是否能回到“恩尼格玛”那一个个转动的齿轮，那台“傅里叶机器”中来回摆动的杠杆：

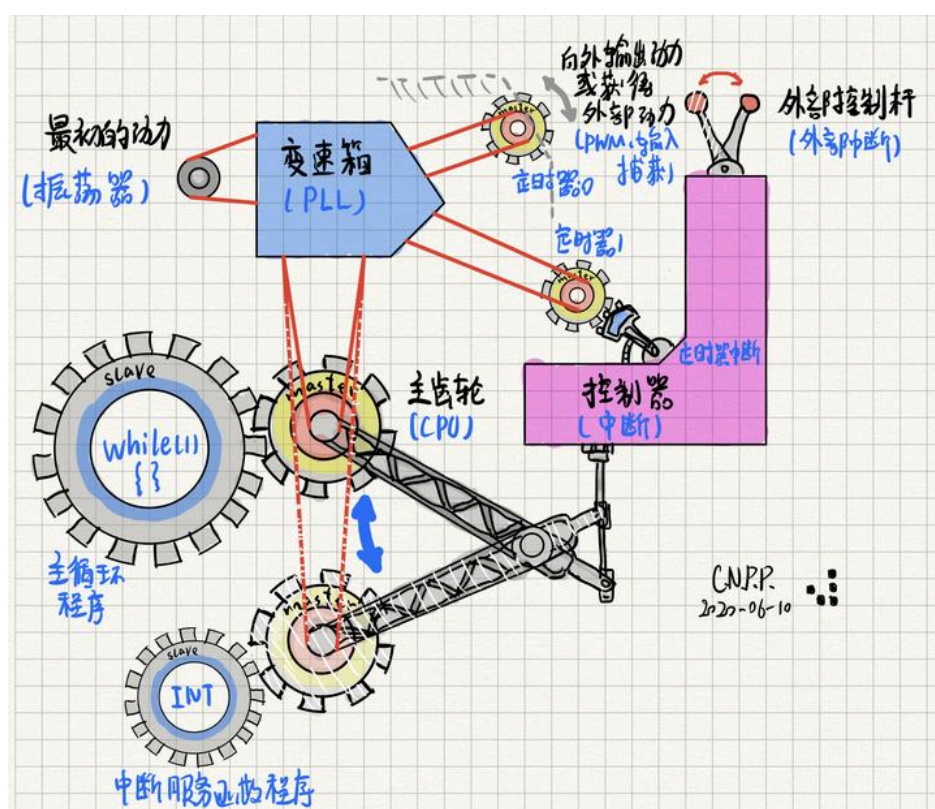


嗯嗯！扯了这么多，其实我是想用齿轮机械来帮助大家形象地了解单片机这个精妙的小装置。首先，我们设想有两种齿轮，一种称为“主动轮”，带有动力，可以自己转动，也可以去驱动其他齿轮；一种称为“从动轮”，只能在主动轮的驱动下转动：



接着，我们想象一只主动轮开始运转，带动从动轮转动，恰似单片机中的核心（**CPU**）开始处理主函数中的内容，从动轮转完一圈，一次while(1){}循环就完成了。

怎么样，是不是挺那么回事（要是我自作多情了就原谅我吧啊啊啊）。那么我们就可以继续想了，这个主动轮转动的频率，是不是就是单片机的核心频率了呢？顺着这个思路往下想，主动轮动力的源泉，似乎就是“晶振”了吧，那这么说，那一阵套复杂的**PLL**、**时钟树**系统，不就是变速箱吗？嗯嗯，于是我画了下面这张图：



可以看到，我为主动轮安排了一个可以平动的设定，这样就可以解释我们前面讲解过的中断：当控制器（**NVIC**）的外部控制杆被搬动时（发生外部中断），控制器把主齿轮移出当前位置（产生中断应答），大从动轮刹车（保存中断现场），接着主动轮移动到从动轮处，开始对其驱动（CPU转而执行中断服务函数中的代码），执行完成后，在控制器的控制下（清除中断标志位），主动轮回到原处（回到中断现场），继续驱动大从动轮。当然我们知道，主动轮的这个位置不止一个，因为我们的单片机通常支持多组中断。

说完这些，大家是不是已经注意到我们的主角了，没错，就是两个小一些的主动轮，即定时器。当然，我们的STM32单片机中是有远多于两个定时器的。从图中，我们可以看出定时器的几个特性：

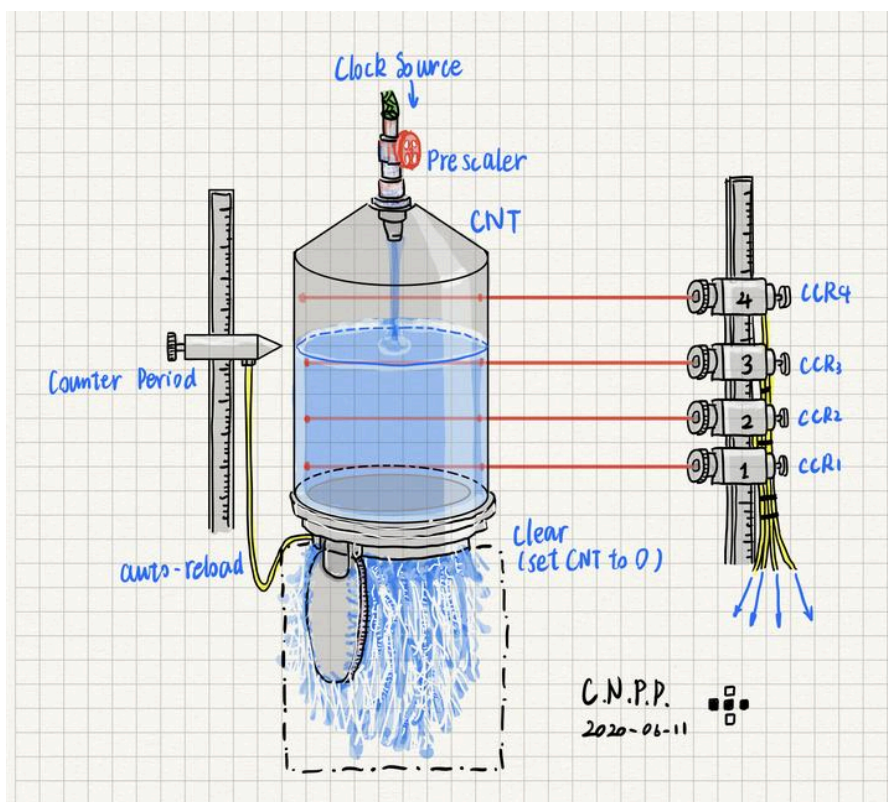
1. 定时器是连接在时钟树上的，振荡器的输出经过倍频、分频，最终驱动定时器运转。
2. 定时器可以发出中断请求，也就是说，当定时器达到某个状态时，可以像外部中断一样，发出请求让CPU去执行相应的中断服务函数。
3. 定时器可以向外输出些什么，也可以从外界感知些什么。

使用这一套齿轮的比喻，其实是想方便大家理解第2节中“时钟频率”与“预分频”两个参数的含义，如果大家在第2节遇到困难，可以返回这里再思考一下。

有了之前的铺垫，也许定时器能好理解一些了？（希望齿轮没事）。那么接下来，我们又要开始类比（涂鸦）了。哎，主要还是因为定时器太重要了，而且需要配置的点有点多，慢慢来吧。

定时器从字面意思看呢，嗯好吧就是定时器，设定时间，到时间后输出这样。不过呢我个人认为，更清晰地解释定时器，换个喻体更好。

定时器在做什么，嗯，在数数。可以正着数，也可以倒着数。我们把数数想象成往一个透明水槽中注水吧，毕竟古人也用过水钟计时。定时器很像这样一套水槽注水系统：



首先看源。定时器是由一个时钟信号驱动的，正如水槽的水源来自上方的管道。注水的速度经由一个阀门控制，同样，时钟信号经过了一个预分频器（Prescaler），频率不变或减慢，再驱动定时器。至于为什么需要预分频器，我们在第2节详述。

水注入槽中，水位就会慢慢升高。同样，定时器**CNT**寄存器中的数值在分频之后的信号的每个上升沿+1（也有可能-1，后面再说）。

看图中，有一个可以上下设定位置的探头“Counter Period”，当水位达到此探头的位置后，探头就会输出控制信号到水槽底部的打阀门，阀门立即开启，迅速泄空槽中的水并自动闭合，开始新一轮的注水循环。那么这个可以设定位置的探头，在定时器中就对应自动装填数值，存储于ARR（Auto Reload Register）寄存器中。也就是说，当**CNT**的值增加到**ARR**的值时，计数器就会发生“向上溢出”，**CNT**自动清零，然后接着从0开始计数（这里描述的是向上计数的情形，还有另外几种模式，后话）。

可想而知，这个**ARR**对于我们的定时器而言是个非常重要的部件。它的数值设定就决定了定时器的循环周期。这里插个题外话，为啥要叫“自动装填”呢？其实在一些老型号的单片机中，比如8051，它内部的三个定时器中的两个是不带有自动装填功能的，当计数溢出后，计数器就“不动”了，需要通过软件代码重新给定时器设定“装填值”，就是我们的ARR值，然后再次启动定时器才行。而我们STM32中的定时器自动装填由硬件电路自动实现，无需人为干预。

这样一个定时器，终于有和我们手机中的“定时器APP”一样的功能了：设定一个周期，然后计时，到时间就发出提醒（定时器溢出时可发出一些信号来驱动其他部件），只不过单片机中的定时器会连续计时罢了。

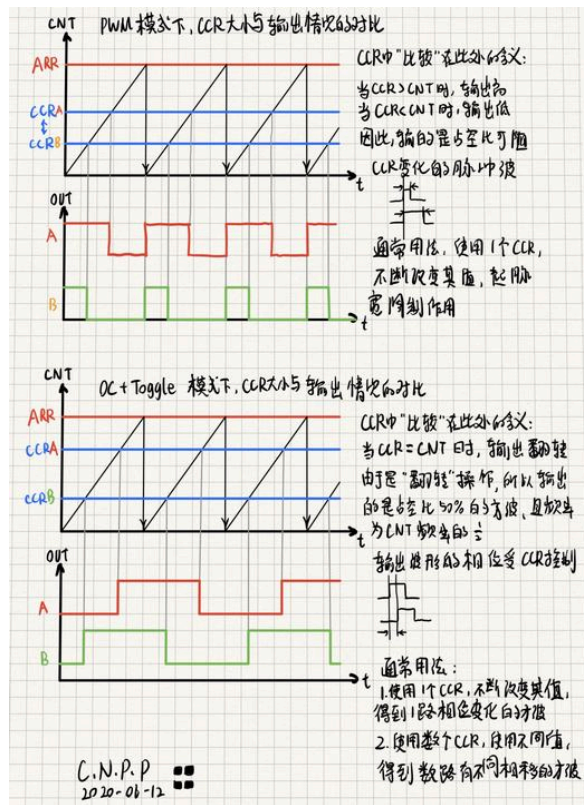
以上就是一个“基础定时器”的结构了。基础定时器没有什么太多的功能，主要就是“计时”，时间一到，通常就是触发一个中断，请求CPU去执行对应中断服务函数中的内容。不过“定时发生中断”这一小小的功能可不得了，往小了说，它可以帮我们做个闪烁周期十分精确的流水灯，往大了说，它可以帮助我们“时分复用”思想设计程序，让单片机中的一个CPU可以应付多个任务。

接下来，就让我们看看工程师们是怎样把这个简单的“蓄水槽”玩出花来的：在右边添加了数个水位传感器。同样，传感器的高度可以自定，一旦水位达到传感器同高处，传感器就会输出控制信号，具体是什么样的信号，可以有所选择。

转到定时器，在我们之前**CLK**、**PSC**、**CNT**、**ARR**四大部件的基础上加入4个捕获/比较寄存器**CCR1~CCR4**，就成为了“普通定时器”（还记得前一个叫做“基本定时器”，吗，其实，还有“高级定时器”，也许会在以后出现）。顾名思义，这四个寄存器与输入捕获以及输出比较两个重要功能有关，在这里我们只说输入捕获。关于输出比较，有兴趣可以看看我之前的帖子

https://www.emoe.xyz/stm32_dma_capture/

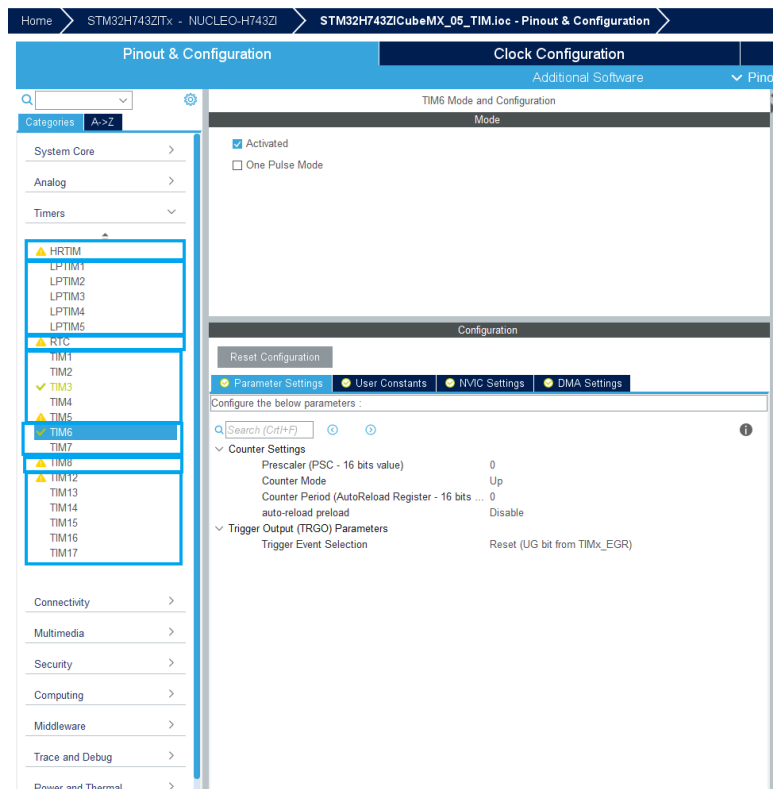
好了，不多说，一图流，介绍一下**CCR**常用的两个功能：



关于PWM调制，可以看下夏老师的这篇帖子https://www.emoe.xyz/ee_tutorial_02/。说了这么多，接下来我们赶紧进入正题，看看STM32的定时器。

2. STM32定时器介绍

首先打开CubeMX，选择你手中的单片机型号，点开左侧Timers界面。这次我的示范是STM32H743芯片，外设非常多，不过还记得吗，在HAL库体系下，32单片机的编程是没有什么区别的。



用鼠标移动到各个定时器上，就能出现相应的说明，在这里简要解释一下：

- **HRTIM**：高精度定时器，自带倍频器，时钟频率可以很高，这样就可以拥有很高的分辨率，同时拥有许多通道以及丰富的触发功能，适合配合ADC使用，完成电机控制、数字电源等精度要求较高的应用。
- **LPTIM**：低功耗定时器，可在各种低功耗模式下运行，甚至可以不依赖内部时钟源运行。在做低功耗的应用时，可以好好研究一下。

以上两种定时器只有部分型号的STM32有，而下面几种STM32都有，不过数量可能不同

- **RTC**：实时时钟，内部由十进制计数器构成，可用于记录时间，也可以输出信号用于唤醒睡眠状态的单片机。用它就可以在我们的作品中添加一个数字钟的功能。**RTC**也常常在低功耗设计中使用。
- **TIM6、TIM7**为基础定时器，主要作用是定时触发中断，或者生成事件（event）来触发**ADC**、**DAC**等外设。基础定时器没有输入和输出相关的功能。
- 除**TIM1**、**TIM8**、**TIM5**、**TIM6**外的**TIMx**，均为普通定时器，它们是在基础定时器的基础上每个定时器带增加了至多4个**CCR**输入、输出通道，可以用于比较输出、输入捕获，还可以配置成正交编码器输入等。当然它们也有触发中断、**DMA**等功能，还可以由一个定时器触发另一个定时器，构成主从定时器。用法十分灵活。
- **TIM1**和**TIM8**为高级定时器，相比普通定时器，增加了更多通道，而且每个通道可以有一对互补波形输出，而且带有插入死区时间以及刹车等功能，很适合用于生成MOS半桥驱动信号。

在使用定时器时，很多时候不仅要依靠HAL库函数，还要手动改寄存器，如前文说的**PSC**、**ARR**、**CCR**等。所以说要多多查阅官方参考手册（讲解真的是很详细），而且有时要翻进HAL库相应函数中看看具体操作的是那个寄存器，这样交叉学习。芯片的参考手册，可以直接在ST官网搜索芯片型号，进入产品页面的Resources页，

The screenshot shows the STMicroelectronics website interface. At the top, there's a navigation bar with 'Products', 'Applications', 'Tools & Software', and 'About ST'. Below this is a breadcrumb trail: 'Microcontrollers & Microprocessors > STM32 32-bit Arm Cortex MCUs > STM32 High Performance MCUs > STM32H7 Series > STM32H743/753 > STM32H743ZI'. The main product page for 'STM32H743ZI' is displayed, featuring a 'Download datasheet' button and a 'Resources' section. The 'Resources' section is divided into four tabs: 'Overview', 'Resources' (selected), 'Tools & Software', and 'Quality & Reliability'. Under the 'Resources' tab, there are three columns of links: 'Quick links' (Product Specifications, Reference Manuals, Design Notes & Tips, Presentations), 'Application Notes' (Application Notes, Programming Manuals, Application Notes for related Tools & Software, Flyers), and 'Technical Notes & Articles' (Technical Notes & Articles, Errata Sheets, HW Model, CAD Libraries & SVD, Brochures). At the bottom, a download progress bar indicates '00 Files selected for download'.

向下翻，找到REFERENCE MANUALS就是。

REFERENCE MANUALS

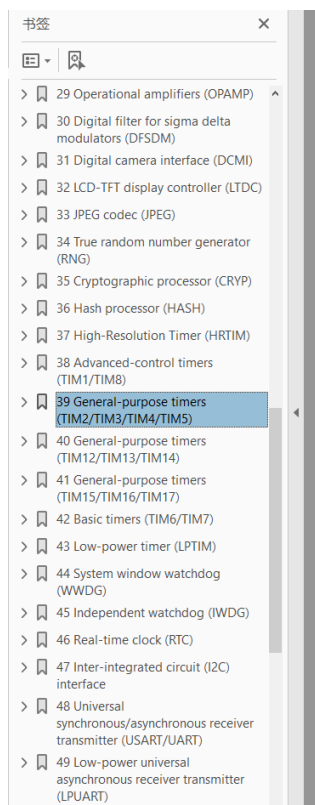
Description	Version	Size	Action
☐ RM0433 STM32H742, STM32H743/753 and STM32H750 Value line advanced Arm®-based 32-bit MCUs	7.0	37.92 MB	PDF

PROGRAMMING MANUALS

Description	Version	Size	Action
☐ PM0253 STM32F7 Series and STM32H7 Series Cortex®-M7 processor programming manual	5.0	5.11 MB	PDF

ERRATA SHEETS

这本手册非常详细，在学习的时候，应该当作字典来查阅使用。



General-purpose timers (TIM2/TIM3/TIM4/TIM5)

RM0433

39 General-purpose timers (TIM2/TIM3/TIM4/TIM5)

39.1 TIM2/TIM3/TIM4/TIM5 introduction

The general-purpose timers consist of a 16-bit/32-bit auto-reload counter driven by a programmable prescaler.

They may be used for a variety of purposes, including measuring the pulse lengths of input signals (*input capture*) or generating output waveforms (*output compare and PWM*).

Pulse lengths and waveform periods can be modulated from a few microseconds to several milliseconds using the timer prescaler and the RCC clock controller prescalers.

The timers are completely independent, and do not share any resources. They can be synchronized together as described in [Section 39.3.19: Timer synchronization](#).

39.2 TIM2/TIM3/TIM4/TIM5 main features

General-purpose TIMx timer features include:

- 16-bit (TIM3, TIM4) or 32-bit (TIM2 and TIM5) up, down, up/down auto-reload counter.
- 16-bit programmable prescaler used to divide (also “on the fly”) the counter clock frequency by any factor between 1 and 65535.
- Up to 4 independent channels for:
 - Input capture
 - Output compare
 - PWM generation (Edge- and Center-aligned modes)
 - One-pulse mode output
- Synchronization circuit to control the timer with external signals and to interconnect several timers.
- Interrupt/DMA generation on the following events:
 - Update: counter overflow/underflow, counter initialization (by software or internal/external trigger)
 - Trigger event (counter start, stop, initialization or count by internal/external trigger)

STM32的定时器有着非常多的细节内容，我们暂时就不涉及了。有机会的话，我们会出一期专门深入STM32定时器应用的文章。接下来，我们通过两个最常用的例子熟悉一下结合HAL库的定时器使用。

3. 实例：定时器中断

学习定时器第一步怎么做？当然是做硬件的“Hello World!”——闪灯了。我们就来让一个灯每秒闪烁5次，还记得最简单做法是什么吗：


```
int main (void) {  
    // 省了略各种初始化函数  
    while (1) {  
        HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin);  
        HAL_Delay(100);  
    }  
}
```

或者：

```
int main (void) {  
    // 省了略各种初始化函数  
    while (1) {  
        HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin, GPIO_PIN_SET);  
        HAL_Delay(100);  
        HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin, GPIO_PIN_RESET);  
        HAL_Delay(100);  
    }  
}
```

那我们现在用定时器中断来实现一下吧，既然要一秒钟闪烁5次，那么就把定时器设置成一秒钟中断5次吧，也就是定时器的频率设为5Hz。嗯，我们先不说具体怎么设定，先看看代码大概写成什么样子。

```

int main (void) {
    // 省了略各种初始化函数
    HAL_TIM_Base_Start_IT(&htim6); // 所使用定时器的初始化函数
    while (1) { // 主循环里设也没有，闪灯在中断回调函数中实现
    }

    // 中断回调函数
    void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
        if (htim == &htim6) { // 如果此中断是TIM6发出的
            HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin, GPIO_PIN_SET);
            HAL_Delay(100);
            HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin, GPIO_PIN_RESET);
            HAL_Delay(100);
        }
    }
}

```

每当中断发生时，CPU就会跑去执行中断回调函数中的闪灯代码了，这样似乎就能完成每秒钟闪烁5次了呢。（中断回调函数，其实就是中断服务函数，为什么换了个名字一会儿再说）

可是，这个程序能正常运行吗？乍一看没问题，但事实上完全无法运行，LD2甚至一直亮着。

注意到了吗，代码在中断回调函数占用了较长的时间，至少200ms，加上函数出入栈的时间，实际上更长。200ms多啊，这对于主频上MHz的内核来说，真的是“度日如年”了。在这漫长的时间里，CPU已经被叫来处理中断了，此时若再有其他中断申请，甚至是同一个中断又发生了（本例中定时器每200ms发出一次中断请求，而处理中断回调函数要超过200ms，一下子就冲了），这样一来，轻则在中断优先级控制器**NVIC**的处理下延后响应后来的中断，重则可能导致逻辑错误（编译器不会帮你检查出来，而程序又确实无法正常运行）。

这种围绕中断等突发事件的代码，就叫做**临界代码**，要非常小心。对于本例中的情况，一般有两种解决方法。

- 第一种，在中断回调函数的开头禁用所有中断，CPU干着事呢，谁也别过来打扰，在中断回调函数结尾再释放中断。此招伤敌一千，自损八百，多一点的中断时间就被阻塞到根本无法及时处理。
- 第二种，也是最好的解决方法之一。首先坚持一个原则：中断回调函数中的内容越少越好，进出中断越快越好。那么我们的任务在哪里运行呢？还是放在主函数中，但要加一个全局变量“FLAG”和一个判断。当判断到FLAG有效时，在主函数中执行对应任务，否则不执行。而中断回调函数则用来设定FLAG有效。

对于第二种方法，下面给出示例：

```

uint8_t FLAG_TASK0; // 事件0标志
uint8_t FLAG_TASK1; // 事件1标志

int main (void) {
    // 省了略各种初始化函数
    HAL_TIM_Base_Start_IT(&htim6);    // TIM6的中断将控制TASK0
    HAL_TIM_Base_Start_IT(&htim7);    // TIM7的中断将控制TASK1
    while (1) {
        if (FLAG_TASK0 == 1) {        // FLAG_TASK0标志触发了
            FLAG_TASK0 = 0;            // 立即清零标志
            TASK0();                    // 运行TASK0
        } else if (FLAG_TASK1 == 1) { // FLAG_TASK1标志触发了
            FLAG_TASK1 = 0;            // 立即清零标志
            TASK1();                    // 运行TASK1
        }
    }
}

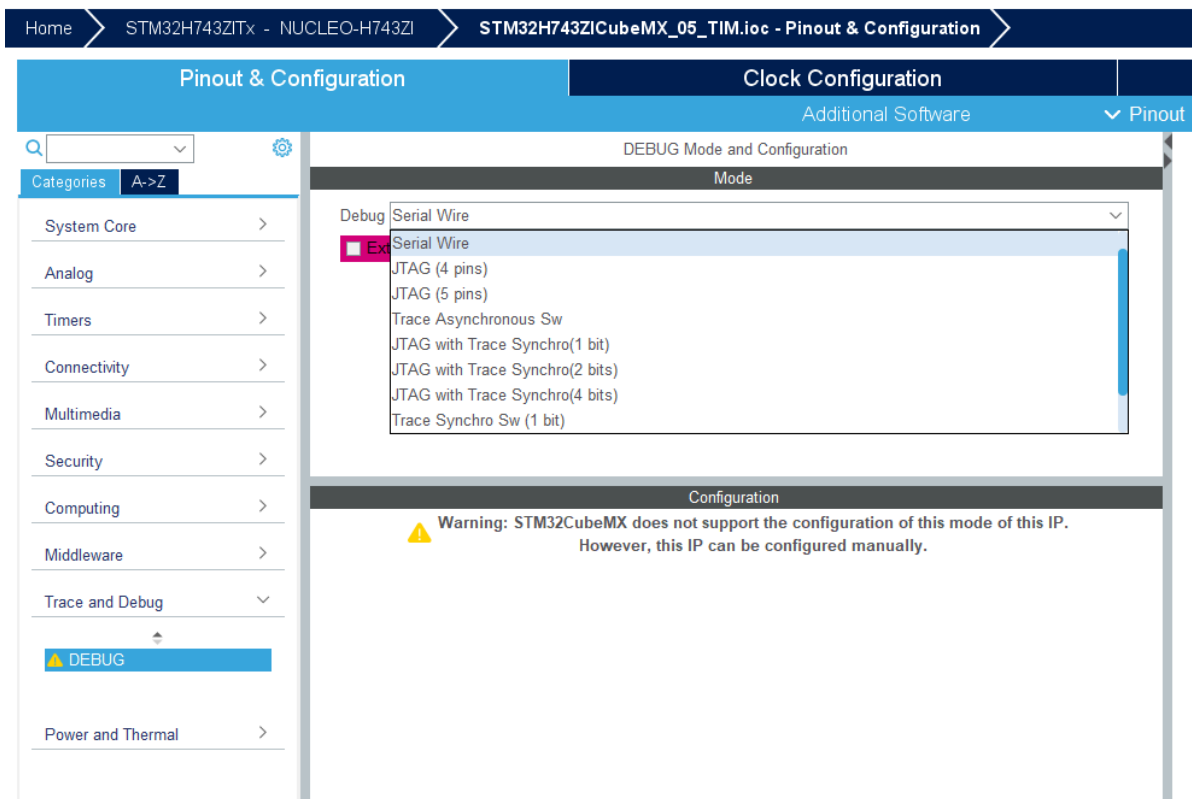
// 中断回调函数
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim == &htim6) {            // 如果此中断是TIM6发出的
        FLAG_TASK0 = 1;                // 将TASK0标志置1
    } else if (htim == &htim7) {      // 如果此中断是TIM7发出的
        FLAG_TASK1 = 1;                // 将TASK1标志置1
    }
}

```

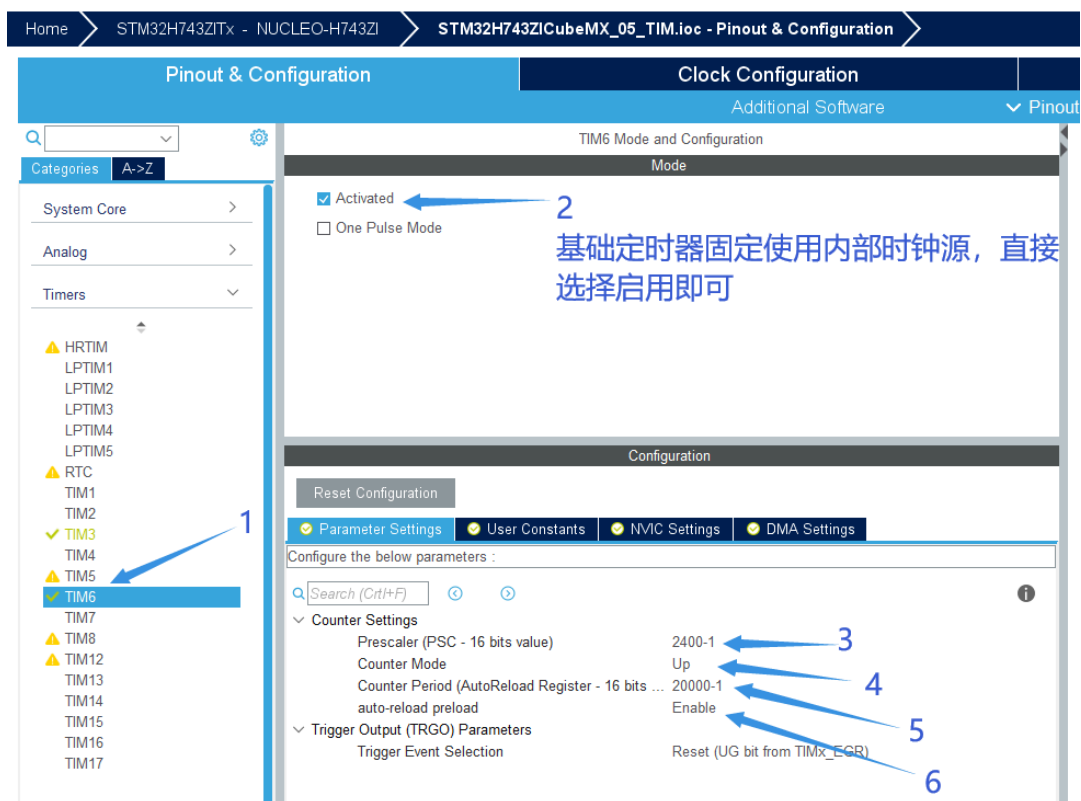
可见，如此以来，中断回调函数中仅做一判断和赋值，可以迅速完成，同时使用了全局标志，使得CPU处理不同中断事件可以有条不紊。其实这里全局标志的使用，就是一种深度为1的FIFO缓冲系统，FIFO缓冲系统常用来在两个运转速度不同的体系间作信息中转，比如此处飞速运行的CPU和不知何时会发生的中断。

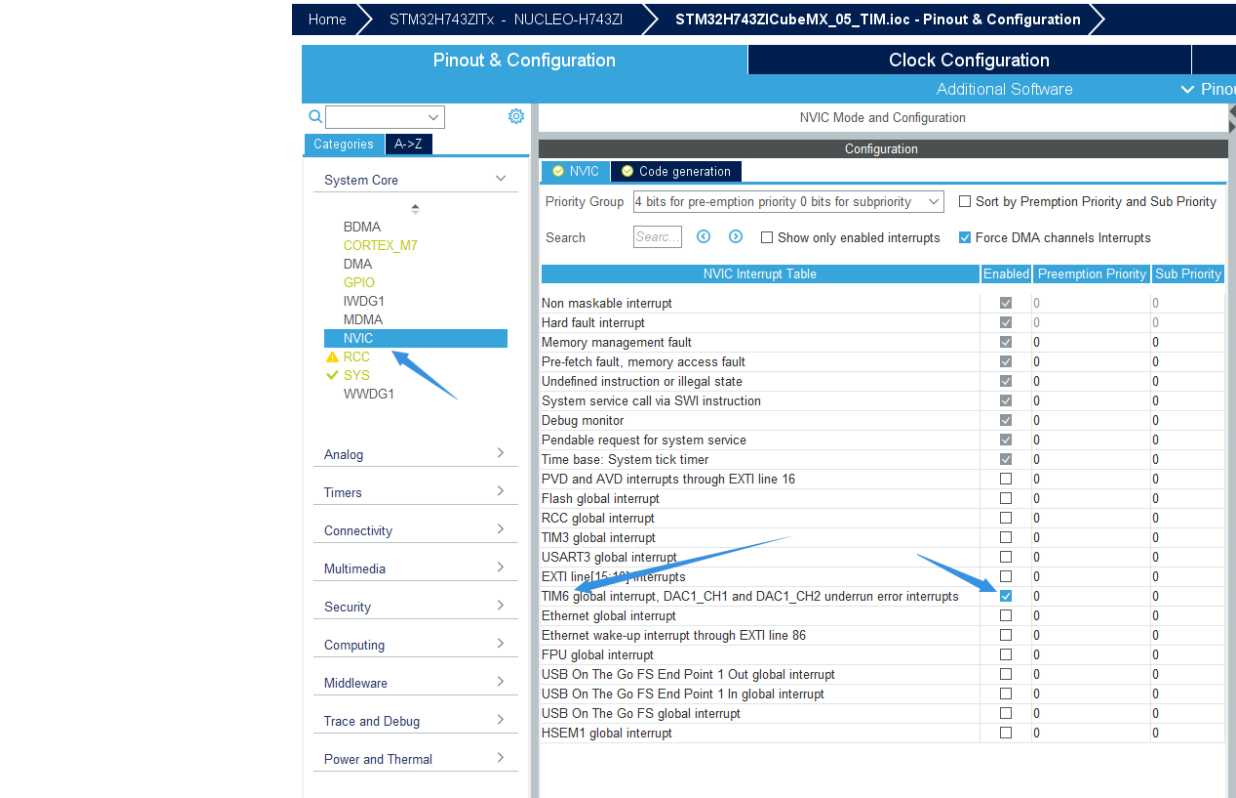
那么思路理清，我们开始真正实战了：

首先，老生常谈的CubeMX基本配置：SWD接口，振荡器选择，时钟树配置。注意部分芯片的调试接口选项不是在System Core 里，而是单独分出来叫做Trace and Debug：



回想基本定时器的配置顺序：时钟源->预分频->重装值->杂项->开启中断：





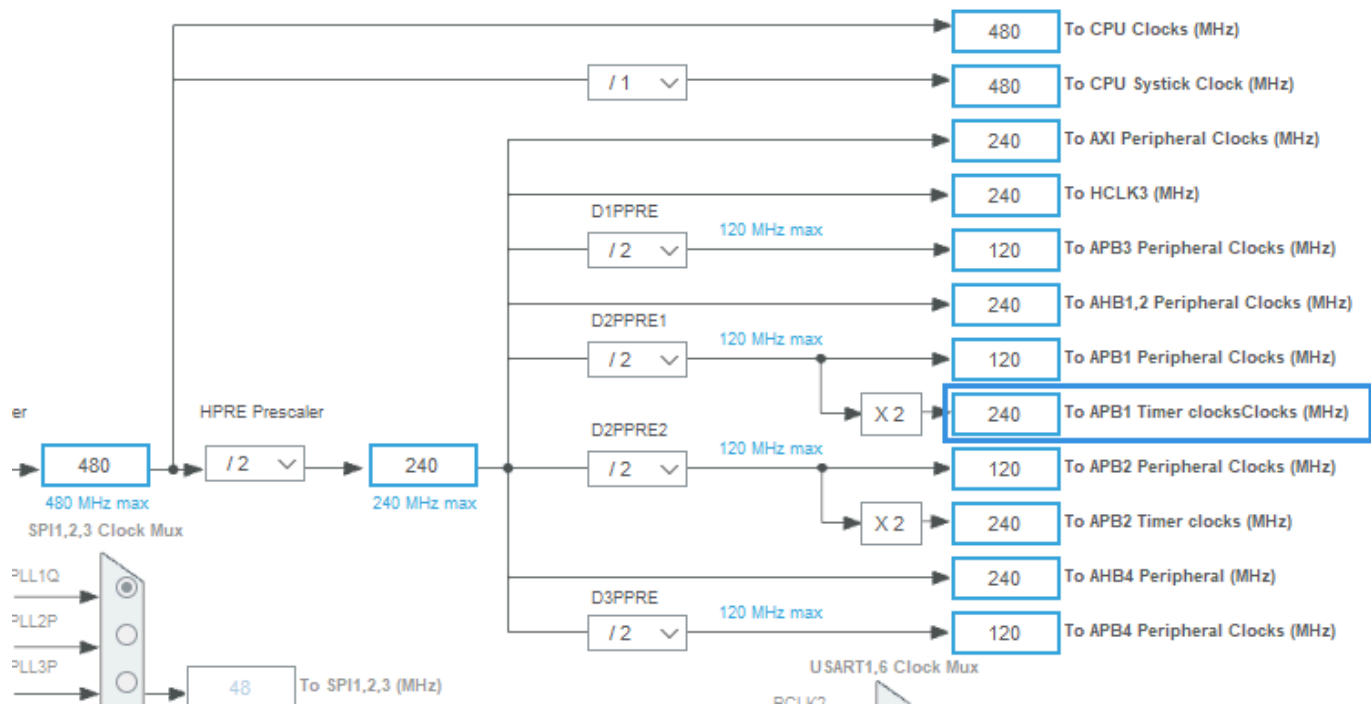
这里有几点需要注意：

- 基本定时器TIM6连接到了内部时钟源上，我们需要知道这个时钟的频率。查阅手册：

Table 8. Register boundary addresses ⁽¹⁾ (continued)			
Boundary address	Peripheral	Bus	Register map
0x4000AC00 - 0x4000D3FF	CAN Message RAM	APB1 (D2)	Section 56.5: FDCAN registers
0x4000A800 - 0x4000ABFF	CAN CCU		Section 56.5: FDCAN registers
0x4000A400 - 0x4000A7FF	FDCAN2		Section 56.5: FDCAN registers
0x4000A000 - 0x4000A3FF	FDCAN1		Section 56.5: FDCAN registers
0x40009400 - 0x400097FF	MDIOS		Section 54.4: MDIOS registers
0x40009000 - 0x400093FF	OPAMP		Section 29.6: OPAMP registers
0x40008800 - 0x40008BFF	SWPMI		Section 53.6: SWPMI registers
0x40008400 - 0x400087FF	CRS		Section 9.8: CRS registers
0x40007C00 - 0x40007FFF	UART8		Section 48.7: USART registers
0x40007800 - 0x40007BFF	UART7		Section 48.7: USART registers
0x40007400 - 0x400077FF	DAC1		Section 26.7: DAC registers
0x40006C00 - 0x40006FFF	HDMI-CEC		Section 59.7: HDMI-CEC registers
0x40005C00 - 0x40005FFF	I2C3		Section 47.7: I2C registers
0x40005800 - 0x40005BFF	I2C2		Section 47.7: I2C registers
0x40005400 - 0x400057FF	I2C1		Section 47.7: I2C registers
0x40005000 - 0x400053FF	UART5		Section 48.7: USART registers
0x40004C00 - 0x40004FFF	UART4		Section 48.7: USART registers
0x40004800 - 0x40004BFF	USART3		Section 48.7: USART registers
0x40004400 - 0x400047FF	USART2		Section 48.7: USART registers
0x40004000 - 0x400043FF	SPDIFRX1		Section 52.5: SPDIFRX interface registers
0x40003C00 - 0x40003FFF	SPI3 / I2S3		Section 50.11: SPI/I2S registers
0x40003800 - 0x40003BFF	SPI2 / I2S2		Section 50.11: SPI/I2S registers
0x40002C00 - 0x40002FFF	Reserved		Reserved
0x40002400 - 0x400027FF	LPTIM1		Section 43.7: LPTIM registers
0x40002000 - 0x400023FF	TIM14		Section 39.4: TIM2/TIM3/TIM4/TIM5 registers
0x40001C00 - 0x40001FFF	TIM13		Section 39.4: TIM2/TIM3/TIM4/TIM5 registers
0x40001800 - 0x40001BFF	TIM12		Section 39.4: TIM2/TIM3/TIM4/TIM5 registers
0x40001400 - 0x400017FF	TIM7		Section 42.4: TIM6/TIM7 registers
0x40001000 - 0x400013FF	TIM6		Section 42.4: TIM6/TIM7 registers
0x40000C00 - 0x40000FFF	TIM5		Section 39.4: TIM2/TIM3/TIM4/TIM5 registers
0x40000800 - 0x40000BFF	TIM4		Section 39.4: TIM2/TIM3/TIM4/TIM5 registers
0x40000400 - 0x400007FF	TIM3		Section 39.4: TIM2/TIM3/TIM4/TIM5 registers
0x40000000 - 0x400003FF	TIM2		Section 39.4: TIM2/TIM3/TIM4/TIM5 registers

1. Accessing a reserved area results in a bus error. Accessing undefined memory space in a peripheral returns zeros.

可以看出TIM6挂载于APB1总线，则其应使用此总线的时钟，在查看CubeMX中的时钟树：



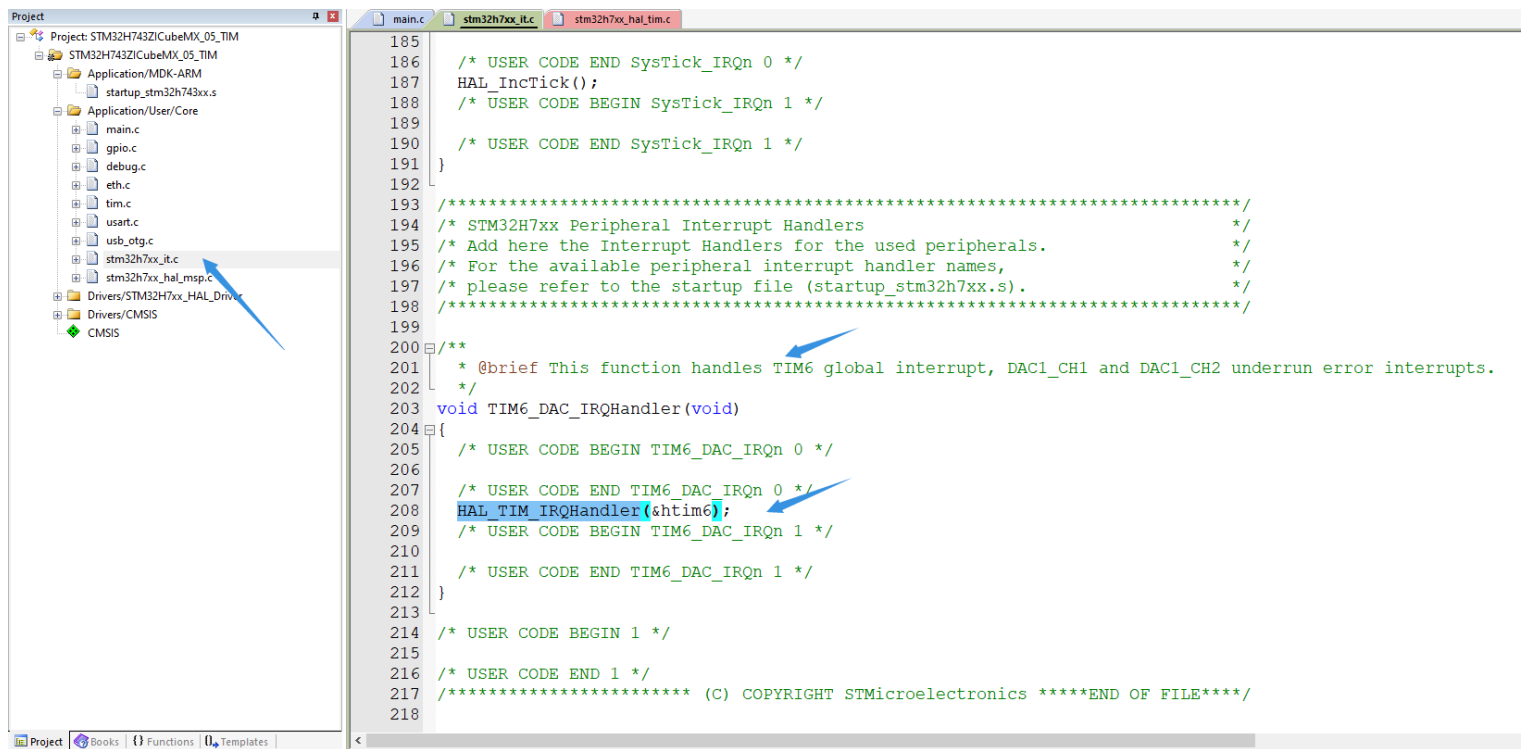
可见APB1上所有定时器的时钟此时为240MHz（注意APB1外设时钟为120MHz）

- 确定时钟源频率后，我们就可以设置**PSC**和**ARR**了。按照之前的要求，我们想让定时器的溢出频率为5Hz，则 $240\text{MHz}/5\text{Hz}=48\text{M}$ 分频。我们知道，一个模值（“容量”）为48M的定时器即可完成此分频，可是我们的**CNT**寄存器只有16位，也就是说模值最大设置为65535，远远不够呀。这就是预分频器**PSC**存在的意义了，“时钟源太快了，**CNT**没有足够的容量来实现较长周期的定时，所以需要预分频器把时钟降慢一些”。所以说，我们把48M拆成 2400×20000 就可以了。注意实际填入**PSC**和**ARR**都有一个“-1”，这是因为定时器是从0开始计数的，由0计到239正好是240次。总结一个定时器频率公式，就是这样：

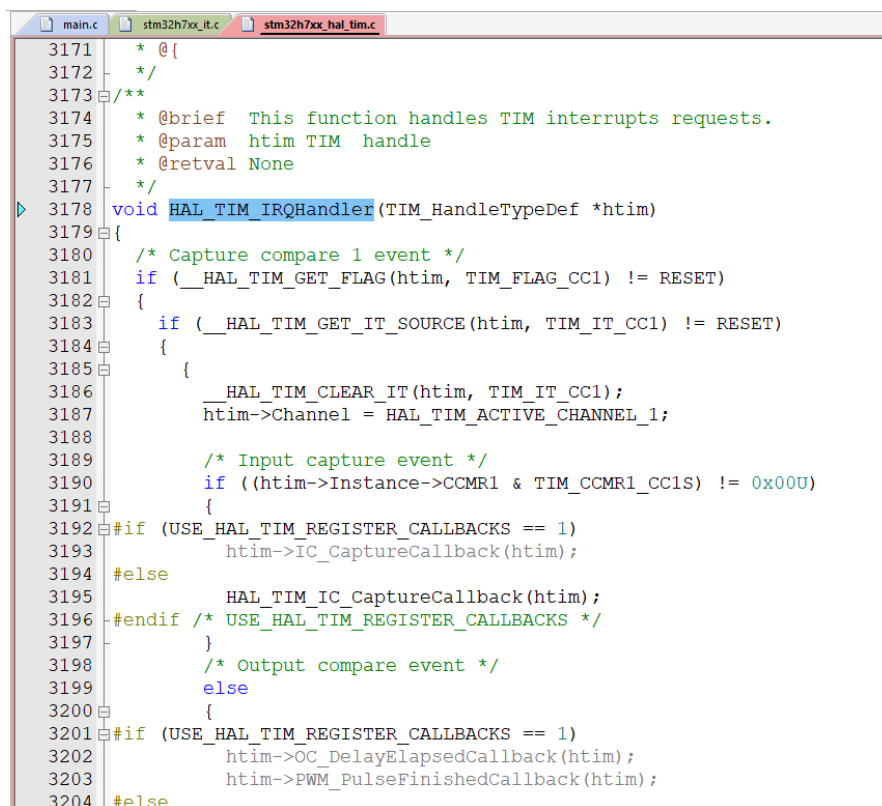
$$f_{\text{TIM}} = \frac{f_{\text{CLK}}}{(PSC+1) \times (ARR+1)}$$

- 解决了这几个主要的，还有些杂项，选项4设定为Up，在这个基础定时器中也只能设定为Up，在普通定时器中，这里还有其他几个选项，比如Up就代表计数器由0增加到ARR，再回到0；Down代表计数器由**ARR**减少到0，再回到**ARR**。而第六个选项并不是指是否自动装填（我们STM32的定时器都是自动装填的），而是在定时器运行过程中，如果**ARR**发生了变化，这个变化是立刻作用，还是在下一次溢出发生后作用。这两点细节，参考手册上有详细的说明和举例，大家可以参考。
- 最后，别忘了去NVIC里把对应的中断打开。由于我们这里只用到了这一处中断，所以中断优先级可以不作设置。其实关于NVIC，我也没有怎么研究过，大家可以多多查阅资料。

生成代码后，我们先看一下给出的文件，找到stm32h7xx_it.c，这里包含了全部我们用到的中断的信息，向下翻找到和TIM6相关的：



这里，我们回顾一下中断是怎样运作的。当中断请求被CPU应答后，CPU就会来处理这个TIM6_DAC_IRQHandler其实，这才是我们说的“中断服务函数”，可以看出，HAL库也在这个函数里给我们留下了写自己代码的空间。不过，为了进一步抽象，HAL库又为不同的中断情况，如溢出，更新等，设立了一套更加具体的中断回调函数，在HAL_TIM_IRQHandler这个函数里选择。我们使用转到定义功能进去看看：



可以看出，这个函数是所有定时器中断通用的，在其内部，再细分到各种中断情况，如图中的HAL_TIM_IC_CaptureCallback。我们呢就找我们需要的——定时器溢出时的中断。往下翻一翻就能在/* TIM Update event */这个注释分条下找到个像那么回事的：HAL_TIM_PeriodElapsedCallback，即“定时器周期消逝回调函数”（中二爆炸），哎其实就是这个。


```
main.c  stm32h7xx_it.c  stm32h7xx_hal_tim.c
3297     HAL_TIM_PWM_PulseFinishedCallback(htim);
3298 #endif /* USE_HAL_TIM_REGISTER_CALLBACKS */
3299     }
3300     htim->Channel = HAL_TIM_ACTIVE_CHANNEL_CLEARED;
3301 }
3302 }
3303 /* TIM Update event */
3304 if (__HAL_TIM_GET_FLAG(htim, TIM_FLAG_UPDATE) != RESET)
3305 {
3306     if (__HAL_TIM_GET_IT_SOURCE(htim, TIM_IT_UPDATE) != RESET)
3307     {
3308         HAL_TIM_CLEAR_IT(htim, TIM_IT_UPDATE);
3309 #if (USE_HAL_TIM_REGISTER_CALLBACKS == 1)
3310         htim->PeriodElapsedCallback(htim);
3311 #else
3312         HAL_TIM_PeriodElapsedCallback(htim);
3313 #endif /* USE_HAL_TIM_REGISTER_CALLBACKS */
3314     }
3315 }
3316 /* TIM Break input event */
3317 if (__HAL_TIM_GET_FLAG(htim, TIM_FLAG_BREAK) != RESET)
3318 {
3319     if (__HAL_TIM_GET_IT_SOURCE(htim, TIM_IT_BREAK) != RESET)
3320     {
3321         HAL_TIM_CLEAR_IT(htim, TIM_IT_BREAK);
3322 #if (USE_HAL_TIM_REGISTER_CALLBACKS == 1)
3323         htim->BreakCallback(htim);
3324 #else
3325         HAL_TIMEx_BreakCallback(htim);
3326 #endif /* USE_HAL_TIM_REGISTER_CALLBACKS */
3327     }
3328 }
3329 /* TIM Break2 input event */
3330 if (__HAL_TIM_GET_FLAG(htim, TIM_FLAG_BREAK2) != RESET)
```

我们再用转到定义看看这个“定时器周期消逝回调函数”：

```
main.c  stm32h7xx_it.c  stm32h7xx_hal_tim.c
5074     * @{
5075     */
5076
5077 /**
5078  * @brief Period elapsed callback in non-blocking mode
5079  * @param htim TIM handle
5080  * @retval None
5081  */
5082 __weak void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
5083 {
5084     /* Prevent unused argument(s) compilation warning */
5085     UNUSED(htim);
5086
5087     /* NOTE : This function should not be modified, when the callback is needed,
5088              the HAL_TIM_PeriodElapsedCallback could be implemented in the user file
5089     */
5090 }
5091
```

原来，这是一个__weak修饰的函数，意思就是，如果用户在自己的文件中以相同的名字定义了这个函数，就会替换掉这里这个函数。这样做的好处是什么呢？可想而知，我们就可以把各种中断回调函数都写进同一个文件里了，而不用找去stm32h7xx_it.c这个生成的文件里写。这样大大提高了我们代码的规范性和可移植性。OK，那我们就直接把这一大段函数定义复制到我们喜欢的地方来用。为了方便，我直接扔到了main.c里。

```

48  /* Private variables -----*/
49
50  /* USER CODE BEGIN PV */
51  uint8_t LED_FLAG;
52  /* USER CODE END PV */
53
54  /* Private function prototypes -----*/
55  void SystemClock_Config(void);
56  /* USER CODE BEGIN PFP */
57
58  /* USER CODE END PFP */
59
60  /* Private user code -----*/
61  /* USER CODE BEGIN 0 */
62  void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
63      if (htim == &htim6) {
64          LED_FLAG = 1;
65      }
66  }
67
68  /* USER CODE END 0 */
69
70  /**
71   * @brief The application entry point.
72   * @retval int
73   */
74  int main(void)
75  {
76      /* USER CODE BEGIN 1 */

```

像这样，定义好全局变量LED标志，然后自己定义回调函数。接着，在初始化部分先清零标志，打开定时器，在主函数中编写响应标志的代码，同时不要忘记及时清零标志。

```

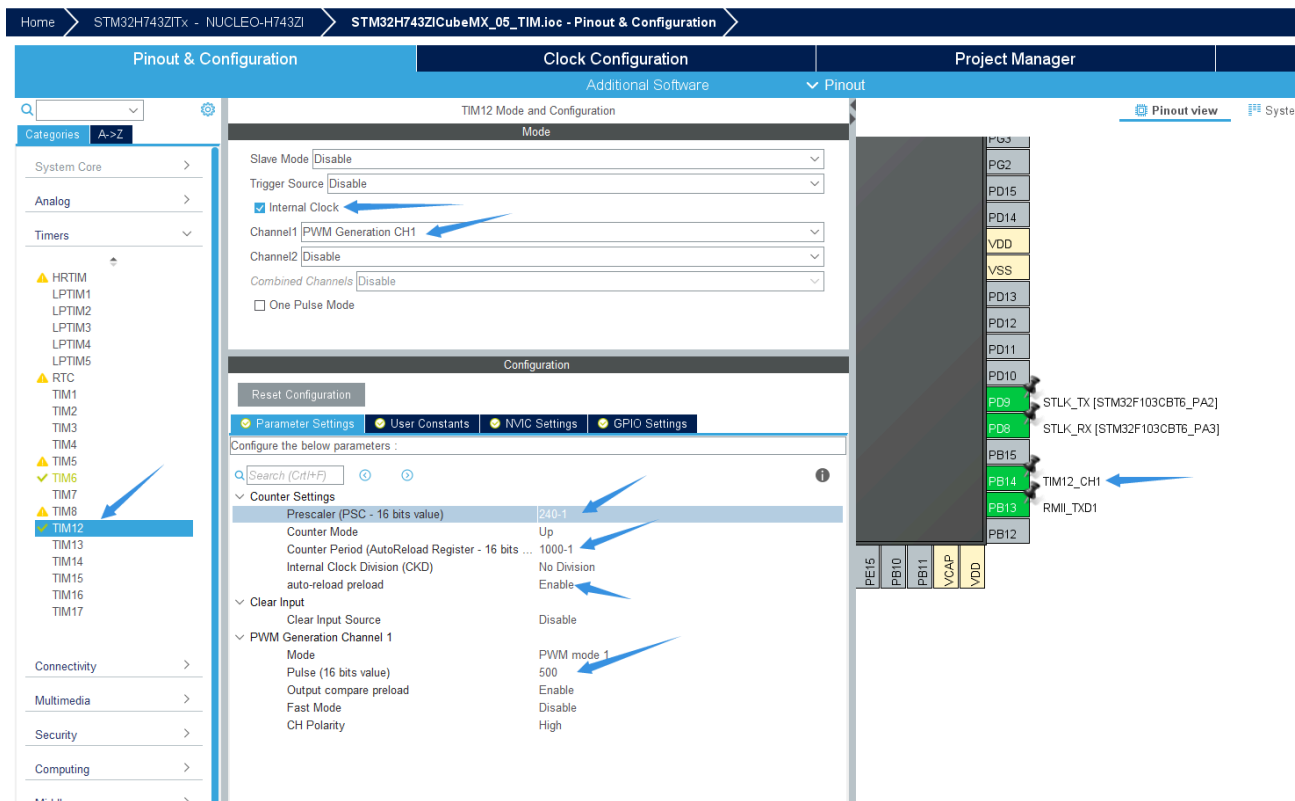
116      LED_FLAG = 0;
117      HAL_TIM_Base_Start_IT(&htim6);
118  /* USER CODE END 2 */
119
120
121
122  /* Infinite loop */
123  /* USER CODE BEGIN WHILE */
124  while (1)
125  {
126      if (LED_FLAG == 1) {
127          LED_FLAG = 0;
128          HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin, GPIO_PIN_SET);
129          HAL_Delay(100);
130          HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin, GPIO_PIN_RESET);
131          HAL_Delay(100);
132      }
133
134  /* USER CODE END WHILE */
135

```

编译，烧录，复位，就可以看到我们的LD2闪烁起来了。

4. 实例：定时器输出PWM

这一部分，我们来学习一下使用普通定时器的一个输出比较功能：PWM。有了前面的铺垫，我们来一图流：



PD14上接了个LED灯。我们希望输出PWM的频率为1kHz，则可以像图上那样设置**PSC**和**ARR**值。通过改变Channel 1的Pulse，即**CCR1**寄存器，可以改变PWM的占空比值，公式为：

$$\text{Duty_x} = \frac{\text{CCR}_x}{\text{ARR}}; x=1,2,3,4.$$

可见，使用高一些的**ARR**，可以提高PWM的分辨率。在这里我们暂且设定为500，在程序中再不断改变CCR1的值来实现呼吸灯的效果。（注意这里有两处preload选项，也跟更新时机有关，希望大家查阅参考手册学习）。在main.c中添加代码如下：

```

int main (void) {
    // 声明流水灯的脉宽以及脉宽变化方向标志
    uint8_t ld1_dir;
    uint16_t ld1_duty;

    // 省了略各种初始化函数

    // 初始化，启动TIM12的PWM模式
    ld1_dir = 0;
    ld1_duty = 0;
    HAL_TIM_PWM_Start(&htim12, TIM_CHANNEL_1);

    while (1) {
        // 判断方向，进行加减
        if (ld1_dir == 0) {
            ld1_duty += 20;
        }
        if (ld1_dir == 1) {
            ld1_duty -= 20;
        }
        if (ld1_duty >= 1000) {
            ld1_dir = 1;
        }
        if (ld1_duty == 0) {
            ld1_dir = 0;
        }

        // 更新CCR1值，直接用寄存器操作
        TIM12->CCR1 = ld1_duty;

        // 延时一下
        HAL_Delay(50);
    }
}

```

这样，呼吸灯就完成了。

6. 后记

呼呼，终于写完了。当初只是接下了“简单介绍一下定时器咋用吧”的坑，没想到忍不住写了这么多。而且，我第一次尝试了一把配上自己画的插图，感觉真的挺刺激啊。

学会使用定时器，真的是学习单片机过程中一个非常重要的分水岭。从这之后，就要接触到各种复杂的多任务，多模块程序设计了。能给大家分享在这里的心得，我真的十分开心。

最后，Happy making, happy coding! .